Echtzeit und API

Lernunterlage: REST, HTTP-GET und -POST, der TuyaClient, Server-Sent Events, Tuya-Push und Docker am eigenen Projekt

1 Das Gesamtbild

Ändert sich ein Gerät, soll das Dashboard es möglichst sofort zeigen. Dafür gibt es zwei Abschnitte, die man auseinanderhalten muss:

```
Gerät → Tuya-Cloud ═ Abschnitt B ═▶ Backend ═ Abschnitt A ═▶ Browser
                     heute: Polling            neu: SSE, sofort
```

> Ehrlicher Stand: Abschnitt A ist gebaut und getestet. Abschnitt B ist weiter Polling (Tür alle 3 Minuten). Echte Tuya-Echtzeit gibt es noch nicht. Der Chip „Live“ meint nur Abschnitt A.

2 Was ist eine REST-API?

Eine API ist eine Schnittstelle, über die Programme miteinander sprechen. **REST** heißt: Es gibt **Ressourcen** mit einer Adresse (Geräte, Räume), und HTTP-Methoden sagen, was damit geschehen soll. Jede Anfrage steht für sich, der Server merkt sich dazwischen nichts über den Client.

Aufruf bei Home Control	Bedeutung	Antwort
GET /api/devices	alle Geräte lesen	200 und JSON-Liste
GET /api/devices/{id}	ein Gerät lesen	200 oder 404
POST /api/devices	Gerät anlegen (nur Admin)	201, sonst 400, 401, 403
PUT /api/devices/{id}	Gerät ändern (nur Admin)	200, sonst 400, 404
DELETE /api/devices/{id}	Gerät löschen (nur Admin)	204 oder 404

3 HTTP-GET und HTTP-POST

	GET	POST
Zweck	lesen	etwas auslösen oder anlegen
Daten	in der Adresse (?date=2026-11-05)	im Body, meist JSON
Verändert etwas?	nein	ja
Wiederholbar?	gefahrlos, Browser und Proxys dürfen es wiederholen	nicht automatisch
Beispiel im Projekt	GET /api/status	POST /api/blink/cameras/{id}/snapshot

Darum ist „Neues Bild“ ein POST: Es verändert etwas (Akku, neues Bild). Bei Tuya ist es genauso: Status lesen ist ein GET, ein Schaltbefehl wäre ein POST. Zu jedem Aufruf gehören Header, z. B. `Authorization: Bearer ...` mit dem Anmelde-Token.

4 Wie der TuyaClient arbeitet

Das Frontend spricht nie direkt mit Tuya. Das Client-Secret bleibt so im Backend. Ablauf für einen Statusabruf:

1. **Token holen:** `TuyaTokenStore` hat ein gültiges Token im Speicher. Sonst holt der Client eines (`GET /v1.0/token?grant_type=1`). Das `SemaphoreSlim` sorgt dafür, dass parallele Anfragen nur einen Abruf auslösen.
2. **Signieren:** `TuyaSignature` bildet HMAC-SHA256 über Client-Id, Token, Zeit, Nonce, Methode, Hash des Bodys und Adresse. Tuya rechnet nach und erkennt jede Veränderung.
3. **Senden:** `GET /v1.0/iot-03/devices/{id}/status` mit den Signatur-Headern.
4. **Auswerten:** Tuya antwortet auch bei Fehlern mit HTTP 200, entscheidend ist `"success"`. Bei einem Token-Fehler (z. B. 1010) wird genau einmal mit neuem Token wiederholt.
5. **Übertragen:** `TuyaDeviceSyncer.Apply` rechnet um (1234 → 123,4 W) und speichert. Ein Ereignis gibt es nur, wenn sich etwas geändert hat.

Testen ohne Internet: Der HttpClient bekommt einen `HttpMessageHandler`. `FakeTuyaHandler` (Tests) und `DemoTuyaHandler` (Demo) antworten statt der Cloud. Alles darüber bleibt echter Code.

5 Server-Sent Events (SSE)

Normalerweise fragt der Browser, und der Server antwortet einmal. Bei **SSE** bleibt die Antwort offen, und der Server schreibt immer wieder etwas hinein. So sieht das auf der Leitung aus:

```
GET /api/live/devices
Authorization: Bearer eyJ...

HTTP/1.1 200 OK
Content-Type: text/event-stream

event: connected
data: null

event: device-change
data: {"deviceIds":["3f2c..."],"source":"tuya-abgleich","atUtc":"2026-11-05T13:06:00Z"}

event: ping
data: null
```

- **Nur Ids, keine Werte:** Der Browser lädt danach über die normalen Endpunkte neu. Es gibt nur einen Weg zu den Daten, und eine verlorene Meldung schadet nicht.
- **ping alle 25 s:** hält die Verbindung durch Proxys offen.
- **Abbruch:** Der Browser verbindet sich neu (1, 2, 4 … 30 s) und lädt danach einmal alles.
- **Tab im Hintergrund:** Verbindung zu, das spart Akku und eine der wenigen Verbindungen des Browsers.
- **Rückfallebene:** Alle Seiten laden weiter jede Minute neu.

Warum SSE und nicht SignalR?

	SSE	SignalR
Richtung	Server → Browser, genau unser Fall	beide Richtungen
Abhängigkeit	keine, in .NET 10 eingebaut	eigene Client-Bibliothek
Token	im Header (fetch)	im Browser meist in der URL

	SSE	SignalR
Erklärung	eine HTTP-Antwort, die nicht aufhört	Hub, Gruppen, Protokoll

Warum fetch statt EventSource? Das eingebaute `EventSource` kann keinen Authorization-Header schicken. Das Token müsste in die URL und stünde dann in Logs.

Publish/Subscribe im Backend

```
// DeviceChangeHub: je Verbindung eine eigene Warteschlange
var channel = Channel.CreateBounded<DeviceChangeEvent>(new BoundedChannelOptions(32)
{
    FullMode = BoundedChannelFullMode.DropOldest,   // voll? Älteste Meldung fällt weg
});

// TuyaDeviceSyncer: erst speichern, dann melden
await db.SaveChangesAsync(ct);
changeHub.Publish(new DeviceChangeEvent(changedIds, DeviceChangeSources.TuyaSync, now));
```

Der Abgleich weiß nicht, wer zuhört, und die Browser wissen nicht, wer die Änderung ausgelöst hat. Das ist die Idee von **Publish/Subscribe**. Ein späterer Tuya-Push wäre nur ein weiterer Absender.

Sicherheit: Die Anmeldung wird nur beim Aufbau geprüft. Deshalb endet der Strom beim Ablauf des Access Tokens (höchstens 15 Minuten). Der Browser verbindet sich neu, und dabei wird wieder geprüft.

6 Wie Push-Nachrichten von Tuya zum Backend kämen

Noch nicht gebaut. Tuya bietet dafür den **Message Service** an, technisch eine Warteschlange auf Basis von **Apache Pulsar**.

Polling (heute)	Push (geplant)
Backend fragt regelmäßig: „Hat sich etwas geändert?“	Tuya legt jede Meldung in eine Warteschlange, das Backend liest mit
Tür alle 3 Minuten, kostet Kontingent, auch ohne Änderung	sofort, kostet je Nachricht
Kurzes Öffnen zwischen zwei Abfragen wird verpasst	jede Meldung kommt an, mit eigener Uhrzeit

- **Die Verbindung geht vom Backend aus** (`pulsar+ssl://mqe.tuyaeu.com:7285`). Kein offener Port, keine öffentliche Adresse. Das geht auch hinter Tailscale und im Container.
- **Bestätigung (Ack):** Pulsar hebt eine Nachricht auf, bis das Backend sie bestätigt. War es offline, kommt sie danach. Doppelte Zustellung ist möglich, die Verarbeitung muss das aushalten.
- **Reihenfolge:** Die Doku nennt keine Garantie. Jede Meldung hat aber eine Zeit, damit erkennt man Veraltetes.
- **Polling bleibt** als Rückfallebene, nur seltener.

> Offene Frage: Laut Doku ist für Gerätenachrichten ein Client über OEM-App oder App SDK vorgesehen. Ob unsere Geräte aus dem verknüpften Smart-Life-Konto Nachrichten liefern, ist nicht belegt. Das klärt erst ein Spike.

7 Wie Docker dazugehört

- **An der API ändert sich nichts:** Dieselben Endpunkte, derselbe Code. Nur die Umgebung ist eine andere.
- **Port:** Im Container hört die App auf 8080, nach außen ist es `127.0.0.1:5400` (Betrieb) oder 5300 (Demo). SSE ist normales HTTP und geht durch diese Weiterleitung.
- **Ausgehend:** Aufrufe an Tuya, später auch Pulsar, gehen aus dem Container hinaus, dafür muss nichts geöffnet werden.
- **Volumes:** Datenbank und Schlüssel liegen außerhalb des Containers und überleben ein Update.
- **Geheimnisse:** kommen beim Start über `.env`, nie ins Image.
- **Grenze:** Blink und der Tuya-Abgleich dürfen nur an einer Stelle laufen. Dienst und Container gleichzeitig mit echten Zugangsdaten verbrauchen doppelt Kontingent, bei Blink melden sie sich gegenseitig ab.

8 Selbst ausprobieren

```
./demo-starten.sh       # Beispielhaus auf http://localhost:5300, keine echten Daten
```

Im Browser anmelden. Die Befehle für das Umschalten der simulierten Tür stehen in docs/ECHTZEIT.md. Die Haustür wechselt dann sofort, ohne Neuladen. Das läuft durch den echten Code, nur die Cloud ist erfunden.

9 Kurzantworten für Rückfragen

Frage	Antwort
Ist das Echtzeit von Tuya?	Nein. Vom Server zum Browser ja, von Tuya zum Server weiter Polling.
Was ist SSE?	Eine HTTP-Antwort, die offen bleibt und in die der Server Ereignisse schreibt.
Warum nicht SignalR?	Nur eine Richtung nötig, SSE ist eingebaut, einfacher, Token bleibt im Header.
Warum stehen keine Werte im Ereignis?	Nur ein Weg zu den Daten, und eine verlorene Meldung schadet nicht.
Was passiert bei Abbruch?	Neu verbinden mit wachsendem Abstand, danach einmal alles laden. Jede Minute lädt die Seite sowieso.
Wie ist der Kanal geschützt?	Nur mit JWT, und er endet beim Ablauf des Tokens.
Unterschied GET und POST?	GET liest und ist gefahrlos wiederholbar, POST löst etwas aus.
Warum geht das Frontend nicht direkt zu Tuya?	Das Secret muss geheim bleiben, und unsere API prüft die Rechte.

10 Merksätze

- Den ganzen Weg betrachten: Ein schneller letzter Abschnitt macht die Daten nicht frischer.
- Nur behaupten, was getestet ist. „Live“ meint den Weg vom Server zum Browser.
- Ereignis sagt: „Es gibt Neues“. Die Daten kommen über den normalen Weg.
- Immer eine Rückfallebene: Polling bleibt.
- Lange Verbindungen brauchen ein Ende: Token-Ablauf, Abmelden, Herunterfahren.